PHASE

18

The ARM Port

Port Turk-OS to 64-bit ARM: exception levels, the GIC, a three-level MMU with two translation bases, a 64-bit clean kernel, and finally Turk-OS 2.0 on three architectures

Stage V progress toward Turk-OS 2.0 (this phase in red) 60 % → 100 %

Duration	5–6 weeks (the Raspberry Pi 4 in Task 18.12 is optional)
Difficulty	★★★★★ (5/5)
You need	Phase 17 complete: the RISC-V port works, so the architecture interface has been tested once; `make ARCH=aarch64 check-generic` from Task 16.9 still passes
You will have	Turk-OS on `qemu-system-aarch64 -machine virt` booting to `tsh` with EL1 exception vectors, GICv2, the generic timer, the PL011 UART, the 39-bit VMSAv8-64 MMU with `TTBR0` and `TTBR1`, `svc` system calls, an ELF64 loader, virtio-blk, `make ARCH=aarch64 test`, and the v2.0 release for three architectures.

1 Why this phase

The RISC-V port proved that the architecture interface of Phase 16 can carry a second machine. But RV32 was kind to you: pointers were still 32 bits, the page tables still had two levels, and the kernel still lived at `0xC0000000`. AArch64 changes everything at once. The instruction set is new, every register and every pointer is 64 bits wide, the MMU has three levels and two base registers, memory has types that you must choose, and the processor reorders memory accesses unless you tell it not to. If the interface survives this port, it is really generic.

This is also the last phase of the plan. You will finish by building one TurkFS image that boots on three architectures, measuring the same benchmarks on all of them, writing down what porting taught you, and tagging Turk-OS 2.0. Expect the MMU (Task 18.6) to take the longest: an AArch64 MMU that is set up slightly wrong usually stops the machine without a word, just as x86 paging did in Phase 6.

2 Concepts you need

2.1 AArch64 in one page

AArch64 has 31 general-purpose 64-bit registers, `x0` to `x30`. Each has a 32-bit view: `w5` is the low half of `x5`, and writing `w5` clears the upper half. Register number 31 means either the stack pointer `sp` or the zero register `xzr`, depending on the instruction. The program counter is not a general register: you change it only with branches. Every instruction is exactly 4 bytes. There is no `push` or `pop`; you move `sp` yourself and store pairs of registers with `stp` and `ldp`. `bl` puts the return address in `x30` (the link register) instead of on the stack, and `ret` jumps to it. The condition flags N, Z, C and V live in `PSTATE`, the processor state. Control registers are **system registers** with names like `VBAR_EL1`; you read them with `mrs` and write them with `msr`. The suffix names the lowest exception level allowed to use the register.

The calling convention is the AAPCS64. Arguments go in `x0`–`x7` and the result comes back in `x0`; `x19`–`x28` are callee-saved; `x29` is the frame pointer and `x30` the link register; `sp` must stay 16-byte aligned. The data model is **LP64**: `int` stays 32 bits, but `long`, `size_t` and every pointer become 64 bits. Table 1 puts the three architectures of Turk-OS side by side.

	i386 (Phases 0–15)	RV32 (Phase 17)	AArch64 (this phase)
data model	ILP32	ILP32	LP64
registers	8 (`eax` … `esp`)	`x1`–`x31`, `x0` = 0	`x0`–`x30`, `sp`, `xzr`
arguments	on the stack (cdecl)	`a0`–`a7`	`x0`–`x7`
callee-saved	`ebx esi edi ebp`	`s0`–`s11`	`x19`–`x28`, `x29`
return address	pushed on the stack	`ra`	`x30`
system call	`int 0x80`, number in `eax`	`ecall`, number in `a7`	`svc #0`, number in `x8`
return from trap	`iret`	`sret`	`eret`
kernel stack on a trap	`esp0` in the TSS	swapped in through `sscratch`	`SP_EL1`, a separate register
page table base	`CR3`	`satp`	`TTBR0_EL1` and `TTBR1_EL1`

Table 1: One kernel, three machines. Only the right-hand column is new in this phase.

2.2 Exception levels

AArch64 replaces x86's rings and RISC-V's modes with four **exception levels** (Figure 1). EL0 runs programs, EL1 an operating system kernel, EL2 a hypervisor and EL3 the secure monitor firmware. An exception can only move execution to the same or a higher level, and `eret` is the only way down. Turk-OS runs at EL1 and its programs at EL0. QEMU's `virt` board has no EL2 or EL3 unless you ask for them, so by default the kernel starts at EL1; with `-machine virt,virtualization=on`, and on a Raspberry Pi 4, it starts at EL2 and must step down.

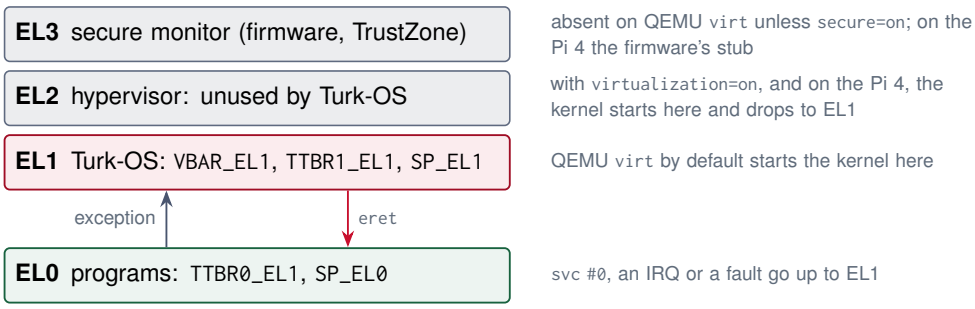


Figure 1: The four exception levels and where Turk-OS uses them.

Each level from EL1 up has its own stack pointer. At EL1 the kernel normally uses `SP_EL1` (the mode is then called EL1h, "h" for handler); `SP_EL0` belongs to the user program. When an exception arrives from EL0, the CPU simply starts using `SP_EL1`, so the hardware itself switches to the kernel stack. There is no TSS to update and no `sscratch` to swap: whatever `sp` held when the kernel last executed `eret` is the kernel stack for the next exception.

2.3 Taking an exception

On every exception to EL1 the CPU saves the return address in `ELR_EL1` and the old `PSTATE` in `SPSR_EL1`, masks all four interrupt classes in `PSTATE.DAIF` (Debug, SError, IRQ, FIQ), switches to `SP_EL1`, and jumps into the **vector table** that `VBAR_EL1` points to. For synchronous exceptions it also writes the reason into `ESR_EL1`, and for aborts the faulting address into `FAR_EL1`. `eret` reloads `PSTATE` from `SPSR_EL1` and the PC from `ELR_EL1`. Unlike x86, the CPU pushes nothing on any stack: saving registers is entirely your code's job.

The vector table is 2 KiB long and must be 2 KiB aligned. It has four groups of four entries, and each entry is 0x80 bytes, room for 32 instructions (Figure 2). The group says where the exception came from; the entry says what kind it is. Turk-OS uses four of the sixteen and treats the rest as fatal.

	synchronous	IRQ	FIQ	SError
current EL, `SP_EL0`	0x000	0x080	0x100	0x180
current EL, `SP_EL1` (kernel)	0x200 faults, `brk`	0x280 device	0x300	0x380
lower EL, AArch64 (user)	0x400 `svc`, faults	0x480 device	0x500	0x580
lower EL, AArch32	0x600	0x680	0x700	0x780

Figure 2: The AArch64 vector table: offsets from `VBAR_EL1`. Coloured entries are the ones Turk-OS handles; grey ones mean a bug (FIQ and SError are not used, and Turk-OS runs no 32-bit code).

EC	Meaning	What Turk-OS does
`0x00`	unknown reason: an undefined instruction, or `hlt` without semihosting	`SIGILL`, or panic in the kernel
`0x07`	SVE, SIMD or floating-point instruction trapped by `CPACR_EL1.FPEN`	`SIGILL`: Turk-OS 2.0 uses no FP (Task 18.2)
`0x15`	`svc` executed in AArch64 state	system call; `ELR_EL1` already points past it
`0x20`, `0x21`	instruction abort from EL0, from EL1	page fault on an instruction fetch
`0x24`, `0x25`	data abort from EL0, from EL1	page fault; ISS bit 6 (WnR) says write
`0x26`	stack pointer alignment fault	`SIGBUS`, or panic: `sp` was not 16-byte aligned
`0x3C`	`brk` executed in AArch64 state	debug print; `ELR_EL1` points at the `brk` itself

Table 2: The exception classes in `ESR_EL1` bits 31:26 that Turk-OS decodes. For aborts, the low six bits (the fault status code) give the type and the table level: `0x04`–`0x07` translation fault, `0x08` plus level access-flag fault, `0x0C` plus level permission fault.

2.4 The GIC and the generic timer

The **Generic Interrupt Controller** (GIC) plays the part of the 8259 PIC and RISC-V's PLIC. Version 2 has two blocks of memory-mapped registers. The **distributor** (`GICD_*`) decides, per interrupt, whether it is enabled, how urgent it is and which CPU receives it. Each CPU has a **CPU interface** (`GICC_*`) through which it acknowledges an interrupt (reading `GICC_IAR` returns its number) and ends it (writing that value to `GICC_EOIR`). Interrupt numbers, called INTIDs, come in three ranges: 0–15 are SGIs, software interrupts between CPUs; 16–31 are PPIs, private to one CPU, such as its timer; 32 and up are SPIs, shared devices such as the UART and the disk. A devicetree `interrupts` property for the GIC has three cells: the type (0 for an SPI, 1 for a PPI), the number within that type, and flags. So SPI 1 is INTID 33. INTID 1023 means "spurious, nothing to do".

The **generic timer** is built into every core, so there is no PIT or SBI call. `CNTFRQ_EL0` holds the counter frequency, `CNTVCT_EL0` the current count, and each timer has a control register (`CNTV_CTL_EL0`: enable and mask bits) and a down-counter (`CNTV_TVAL_EL0`) that raises its PPI when it reaches zero. Turk-OS uses the **virtual** timer, INTID 27, which Linux also uses at EL1; the physical EL1 timer is INTID 30. Read the frequency, never assume it: QEMU gives a Cortex-A72 62.5 MHz, while newer CPU models get 1 GHz.

2.5 Translation with two base registers

With the 4 KiB granule and 39-bit virtual addresses (`T0SZ` = `T1SZ` = 25 in `TCR_EL1`), a walk starts at level 1 and has three levels of 512 eight-byte entries, each table exactly one page (Figure 3). Bits 38:30 of the address select the level-1 entry, 29:21 the level-2 entry, 20:12 the level-3 entry, and 11:0 the byte. Bits 63:39 must be all zeros or all ones. All zeros means the walk starts at `TTBR0_EL1`; all ones means `TTBR1_EL1`; anything else faults. So the 64-bit space has a 512 GiB bottom half for one process, a 512 GiB top half for the kernel, and an enormous hole between them (Figure 4).

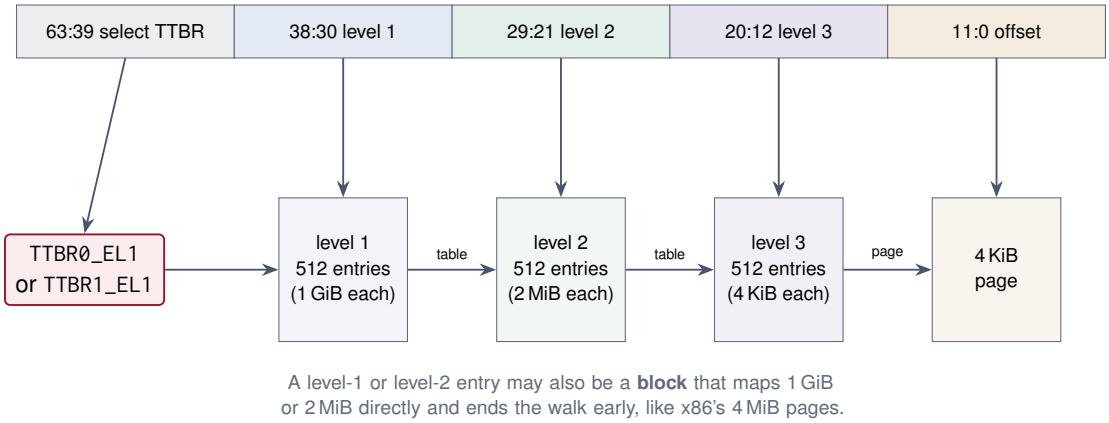


Figure 3: VMSAv8-64 translation of a 39-bit virtual address with 4 KiB pages. As on x86, every table pointer is a physical address.

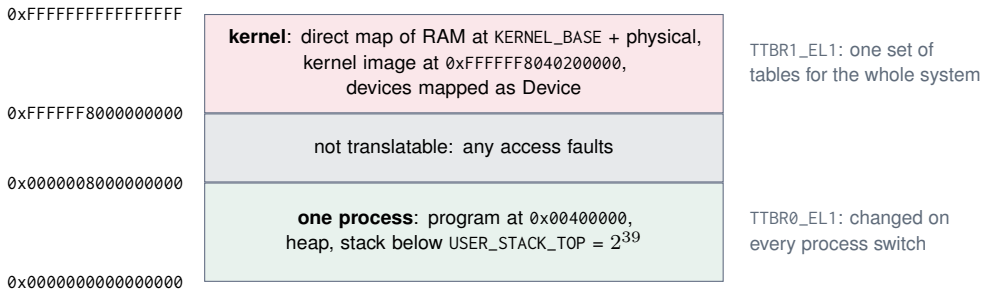


Figure 4: The Turk-OS address space on AArch64. Compare Phase 10's Figure: the user half no longer has to share a page directory with the kernel.

This split removes a chore from x86 and RISC-V, where every new page directory needed copies of the kernel's entries. On AArch64 a process's level-1 table contains only user mappings, and `arch_mmu_switch` changes only `TTBR0_EL1`. Table 3 compares the leaf entries of the three MMUs you have now programmed.

2.6 Memory types and barriers

On x86 and in QEMU's RISC-V board you never had to say what kind of memory an address was. On AArch64 every mapping names a **memory type** through its AttrIndx field, which selects one of eight attribute bytes in `MAIR_EL1`. Turk-OS needs two. **Normal** memory (attribute `0xFF`: write-back cacheable) may be cached, read speculatively, and have its accesses merged and reordered: right for RAM, disastrous for a device, where a read can pop a byte out of a FIFO and the order of writes is the protocol. **Device-nGnRnE** memory (attribute `0x00`) forbids all of that: no gathering of accesses, no reordering, no early write acknowledgement. Every MMIO region must be mapped as Device. With the MMU off, the CPU treats every data access as Device-nGnRnE, which is why early boot code runs without caches and must not make unaligned accesses.

The CPU is also free to reorder ordinary loads and stores as seen by other observers, such as another core, a DMA engine or the MMU's own table walker. Three barrier instructions restore order. `dmb` (data memory barrier) keeps memory accesses before it ahead of those after it. `dsb` (data synchronisation barrier) waits until earlier accesses, and TLB and cache maintenance, have actually completed. `isb` (instruction synchronisation barrier) throws away the instructions already fetched, so the next ones see a just-written system register. An option names the scope: `ish` is the inner shareable domain (all the cores), `st` restricts it to stores. So changing a live page-table entry is: write it, `dsb ishst` so the walker can see it, `tlbi` to drop the old translation, `dsb ish` to wait for that, and `isb`.

Property	x86 PTE (Phase 6)	Sv32 PTE (Phase 17)	AArch64 page descriptor
valid	P, bit 0	V, bit 0	bit 0, with bit 1 = 1 for a page
writable	R/W, bit 1	W, bit 2 (R, bit 1, readable)	AP[2], bit 7, set means *read-only*
user	U/S, bit 2	U, bit 4	AP[1], bit 6
executable	always, without PAE	X, bit 3	UXN bit 54, PXN bit 53, set means *not* executable
accessed	A, bit 5, set by hardware	A, bit 6	AF, bit 10; if clear, the first access faults
dirty	D, bit 6, set by hardware	D, bit 7	none in Armv8.0: map read-only and catch the fault
global	G, bit 8	G, bit 5	nG, bit 11, set means *not* global
cache type	PCD, PWT (bits 4, 3)	none: fixed by the platform	AttrIndx, bits 4:2, an index into `MAIR_EL1`; SH, bits 9:8
frame	bits 31:12	PPN, bits 31:10	bits 47:12

Table 3: Three page-table entry formats. Note how many AArch64 bits are inverted: they are set to take a right away, not to grant one.

The access flag

A descriptor whose AF bit is 0 is valid but raises an access-flag fault the first time it is used. Armv8.0 never sets AF in hardware (later versions can, if `TCR_EL1.HA` asks for it). An operating system can clear AF on purpose to learn which pages are in use, which is the information OSTEP's clock algorithm needs. Turk-OS simply sets AF in every descriptor it writes; forgetting to do so is the classic first bug of an AArch64 MMU.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	OSC	§9.7 *Example: ARMv8 Architecture*	383–384	Granules, levels and regions in two pages, after the IA-32 example you read in Phase 6.
REQUIRED	ARM	*Learn the architecture: AArch64 Exception Model* (exception levels, exception types, the vector table, handling and returning)	–	Tasks 18.2, 18.4 and 18.9.
REQUIRED	ARM	*Learn the architecture: AArch64 memory management* (translation regimes, granules, TTBR0/TTBR1, `TCR_EL1`, TLB maintenance)	–	Task 18.6, before you write a line of it.
RECOMMENDED	ARM	*Learn the architecture: AArch64 memory attributes and properties*	–	Normal versus Device memory, shareability, MAIR.
RECOMMENDED	ARM	*Learn the architecture: Memory Systems, Ordering, and Barriers*	–	What `dmb`, `dsb` and `isb` really guarantee.
RECOMMENDED	ARM	*Learn the architecture: Generic Timer*	–	Counter, frequency, the timer registers (Task 18.5).

Priority	Book	What to read	Pages	Why now
RECOMMENDED	ARM	*ARM Generic Interrupt Controller Architecture Specification*, version 2.0 (IHI 0048B), Ch. 4 *Programmers' Model*	–	The distributor and CPU interface registers of Task 18.5.
RECOMMENDED	SPEC	QEMU documentation, *'virt' generic virtual platform* (Arm), especially *Hardware configuration information for bare-metal programming*	–	What the board guarantees and where it puts the devicetree (Task 18.1).
RECOMMENDED	ARM	*Semihosting for AArch32 and AArch64*: the trap instructions and `SYS_EXIT (0x18)`	–	The test exit of Task 18.10.
RECOMMENDED	OSC	§17.3 *Protection Rings*, the ARM part (TrustZone, exception levels)	669–671	Why EL2 and EL3 exist above your kernel.
RECOMMENDED	OSTEP	§28.10 *Load-Linked and Store-Conditional*	311–312	Exactly what `ldaxr/stlxr` do (Task 18.8).
RECOMMENDED	OSTEP	§19.5 *TLB Issue: Context Switches*	200–202	Flushing on every switch versus ASIDs, for `TTBR0_EL1`.
OPTIONAL	ARM	DDI 0487, the chapters *The AArch64 System Level Programmers' Model* and *The AArch64 Virtual Memory System Architecture*	–	The authoritative answer when a guide is vague.
OPTIONAL	ARM	*Procedure Call Standard for the Arm 64-bit Architecture* (AAPCS64)	–	Register roles and stack alignment, for the assembly and the user ABI.
OPTIONAL	MOS	§12.3.9 *Useful Techniques*: *Hiding the Hardware*	1005–1007	Ideas for the porting chapter of Task 18.11.
OPTIONAL	MOS	§11.3.1 *Operating System Structure*, the part *The Hardware Abstraction Layer*	880–882	How Windows separates the machine from the kernel.
OPTIONAL	LKP	Ch. 3, *Kernel build for the Raspberry Pi*	128–135	Boot files and cross-building for the Pi of Task 18.12.

Keep Linux's *Documentation/arch/arm64/booting.rst* at hand as well: it defines the Image header and the CPU state that QEMU and the Raspberry Pi firmware both produce for Task 18.2.

Real-system lens: how Linux arm64 does it

Linux enters at `primary_entry` in `arch/arm64/kernel/head.S`, whose `init_kernel_el` drops from EL2 to EL1 when the kernel does not stay there itself. `__cpu_setup` in `arch/arm64/mm/proc.S` computes `MAIR_EL1` and `TCR_EL1` much as your Task 18.6 does, before `head.S` turns the MMU on. The vector table is `vectors` in `arch/arm64/kernel/entry.S`: sixteen uses of the `kernel_ventry` macro, each saving registers into a `struct pt_regs` and calling C handlers such as `el0t_64_sync_handler` in `entry-common.c`, which sends `svc` to `el0_svc`. For contrast, MINIX 3's ARM port (`minix/kernel/arch/earm` in the MINIX 3 source tree, not in OSDI) targets 32-bit ARMv7 boards such as the BeagleBone. Its `exc.S` has the old ARM vector table: eight single `ldr pc` instructions, one per exception type, instead of sixteen 128-byte slots.

4 Build it

Task 18.1 Meet the board

QEMU's `virt` board for Arm is documented as having only two fixed addresses: flash at 0 and RAM at `0x40000000`. Everything else may move between QEMU versions, so the kernel finds it in the devicetree, as on RISC-V. Pass `-cpu cortex-a72`: without it QEMU emulates a 32-bit Cortex-A15. Ask for GICv2 explicitly too; it is the default today only because you use eight or fewer CPUs. Dump the devicetree (keep a copy in `tests/data/` for the FDT tests of Task 16.10) and look at the memory map.

terminal

```
qemu-system-aarch64 -machine virt,gic-version=2,dumpdtb=build/aarch64/virt.dtb -cpu cortex-a72 -m 128M
dtc -I dtb -O dts build/aarch64/virt.dtb | less
printf 'info mtree -f\nquit\n' | qemu-system-aarch64 -machine virt,gic-version=2 -cpu cortex-a72 \
    -display none -monitor stdio -S
```

Device	Address (QEMU 10)	Interrupt
GICv2 distributor, CPU interface	`0x08000000`, `0x08010000`	–
PL011 UART	`0x09000000`	SPI 1 = INTID 33
virtio-mmio, 32 slots	`0x0a000000` + `0x200` $\times n$	SPI $16 + n$ = INTID $48 + n$
RAM	`0x40000000`	–
generic timer	in the core	virtual: INTID 27; EL1 physical: INTID 30

QEMU starts a kernel in one of two ways. An ELF file is booted "bare-metal": the CPU starts at the ELF entry at the highest exception level, `x0` holds nothing useful, and the devicetree waits at the start of RAM. Any other file is booted with the **Linux boot protocol**: `x0` holds the devicetree's physical address, an `-initrd` is loaded and listed in `/chosen`, and the CPU starts at EL1, or at EL2 if the board has it. The Raspberry Pi firmware does the same, so Turk-OS uses the Linux protocol on both.

Verify: The decoded tree shows `memory@40000000` with 128 MiB, `pl011@9000000` with `interrupts = <0x00 0x01 0x04>`, an `intc@8000000` compatible with `arm,cortex-a15-gic` (QEMU's name for a GICv2), a `timer` node with four PPIs, and 32 `virtio_mmio` nodes. Note in `docs/porting.md` which of them your code reads.

Task 18.2 The first instructions

`boot.S` starts with the 64-byte **arm64 Image header**, which makes the raw binary bootable by QEMU and by the Pi firmware. With `text_offset` 0, both load it 2 MiB above a 2 MiB-aligned base, that is at `0x40200000` on QEMU. Then core 0 continues and any other core parks. On QEMU the other cores are not even running: they start powered off and wait for a PSCI `CPU_ON` call (a firmware call through `hvc` or `smc`) that Turk-OS never makes. On the Pi they wait in the firmware's spin loop. Checking `MPIDR_EL1` costs three instructions and protects you on any board.

If `CurrentEL` says EL2, prepare EL1 and step down with `eret`: `HCR_EL2.RW` makes EL1 a 64-bit level, `CNTHCTL_EL2` lets EL1 use the timers, `SCTLR_EL1` gets a known value with the MMU off, and `SPSR_EL2` describes the state to "return" to: EL1h with all interrupts masked.

kernel/arch/aarch64/boot.S

```
// kernel/arch/aarch64/boot.S -- entered at EL2 or EL1, MMU off, x0 = FDT address
    .section .text.boot, "ax"
    .global _start
_start:                             // the 64-byte arm64 Image header comes first
    b     primary                   // code0: jump over the rest of the header
    .word 0                         // code1
```

```
    .quad 0                         // text_offset: 2 MiB above a 2 MiB-aligned base
    .quad _kernel_size              // image_size, defined in kernel.ld
    .quad 0x2                       // flags: little-endian, 4 KiB pages
    .quad 0, 0, 0                   // reserved
    .word 0x644d5241                // magic "ARM\x64"
    .word 0                         // reserved

primary:
    mov   x19, x0                   // keep the FDT pointer safe
    mrs   x1, mpidr_el1
    and   x1, x1, #0xff             // Aff0: this core's number in the cluster
    cbnz  x1, park                  // only core 0 boots Turk-OS
    mrs   x1, CurrentEL
    lsr   x1, x1, #2                // CurrentEL[3:2] holds the exception level
    cmp   x1, #2
    b.ne  at_el1
    mov   x1, #(1 << 31)            // HCR_EL2.RW = 1: EL1 runs in AArch64
    msr   hcr_el2, x1
    mov   x1, #3                    // CNTHCTL_EL2: EL1PCTEN | EL1PCEN, timers usable at EL1
    msr   cnthctl_el2, x1
    msr   cntvoff_el2, xzr          // virtual count = physical count
    ldr   x1, =0x30d00800           // SCTLR_EL1: MMU and caches off, RES1 bits set
    msr   sctlr_el1, x1
    mov   x1, #0x3c5                // SPSR_EL2: D, A, I, F masked; mode EL1h
    msr   spsr_el2, x1
    adr   x1, at_el1
    msr   elr_el2, x1
    eret                            // "return" into EL1 at at_el1
```

At EL1, select `SP_EL1`, set `CPACR_EL1` to 0, give the CPU a stack, clear `.bss`, store the devicetree pointer where `arch_early_init` will find it, and call `kmain`. Every address here comes from `adrp`, which is PC-relative, so the code works at the physical address it was loaded at.

kernel/arch/aarch64/boot.S (continued)

```
at_el1:
    msr   spsel, #1                 // the kernel uses SP_EL1, not SP_EL0
    msr   cpacr_el1, xzr            // FPEN = 00: any FP/SIMD instruction traps (EC 0x07)
    adrp  x1, boot_stack_top
    add   x1, x1, :lo12:boot_stack_top
    mov   sp, x1                    // adrp is PC-relative, so this is a physical address
    adrp  x1, __bss_start
    add   x1, x1, :lo12:__bss_start
    adrp  x2, __bss_end
    add   x2, x2, :lo12:__bss_end
1:  cmp   x1, x2
    b.hs  2f
    str   xzr, [x1], #8             // .bss is 8-byte aligned in kernel.ld
    b     1b
2:  adrp  x1, boot_fdt
    str   x19, [x1, :lo12:boot_fdt] // arch_early_init() reads it
    bl    kmain
park:
    wfe                             // other cores sleep here for ever
    b     park
```

No floating point in Turk-OS 2.0

Compilers use the 32 SIMD registers for ordinary code too: `memcpy`, structure copies, vectorised loops. If the kernel did, every system call would silently destroy user registers. So the kernel is built with `-mgeneral-regs-only`, and `CPACR_EL1.FPEN` (bits 21:20) stays `00`, so any FP or SIMD

instruction traps with EC `0x07`. The userland is built with `-mgeneral-regs-only` as well, since Turk-libc has no floating point. A program that uses FP anyway gets `SIGILL` instead of corrupting another process's registers. Real FP support (`FPEN` = `0b11`, and saving `q0`–`q31`, `FPCR` and `FPSR` per task) is a stretch goal.

The linker script puts `.text.boot` first and defines `__bss_start`, `__bss_end`, `_end` and `_kernel_size = _end - _start`; `arch.mk` turns the ELF into the raw image. Until Task 18.6 the MMU stays off, so link the kernel at its physical address (`KERNEL_BASE` 0 in `memlayout.h`) and add `-mstrict-align`, because Device memory faults on unaligned accesses.

kernel/arch/aarch64/arch.mk

```
CROSS       := aarch64-elf-
ARCH_CFLAGS := -march=armv8-a -mgeneral-regs-only -mstrict-align   # drop -mstrict-align in Task 18.6
ARCH_SRCS   := $(wildcard kernel/arch/aarch64/*.c kernel/arch/aarch64/*.S) kernel/drivers/pl011.c \
               kernel/drivers/virtio_mmio.c kernel/drivers/virtio_blk.c
LDSCRIPT    := kernel/arch/aarch64/kernel.ld
QEMU        := qemu-system-aarch64
QEMU_FLAGS  := -machine virt,gic-version=2 -cpu cortex-a72 -m 128M -nographic \
               -kernel $(BUILD)/kernel8.img -global virtio-mmio.force-legacy=false \
               -drive if=none,format=raw,file=$(BUILD)/turkfs.img,id=hd0 -device virtio-blk-device,drive=hd0
QEMU_TEST_FLAGS  := -semihosting-config enable=on,target=native -append test
TEST_PASS_STATUS := 0              # QEMU's exit status when all tests pass (i386: 1)

$(BUILD)/kernel8.img: $(BUILD)/kernel.elf
    $(CROSS)objcopy -O binary $< $@
```

Verify: `make ARCH=aarch64 debug`, then in `gdb-multiarch build/aarch64/kernel.elf`: `x0` at `_start` holds an address whose first word reads `0xedfe0dd0` (the FDT magic `0xd00dfeed`, big-endian), `CurrentEL` is `0x4`, and `kmain` is reached. Run once more with `-machine virt,virtualization=on`: `info registers` in the QEMU monitor first shows `EL2h`, later `EL1h`.

Task 18.3 A new UART: the PL011

The `virt` board's console is an Arm PL011, not a 16550, so it gets its own driver next to the shared 16550 core of Task 16.8. It registers with `console_register()` like any other console, and its receive interrupt feeds `tty_input_char()`. The baud rate divisor is clock/(16 × baud), split into an integer part and a fractional part in 64ths; read the clock from the devicetree (QEMU's `apb-pclk` node says 24 MHz), because the Pi's is different. QEMU's model ignores the divisors, real hardware does not.

kernel/drivers/pl011.c

```
#define UART_DR     0x000               /* data: write to send, read to receive */
#define UART_FR     0x018               /* flags                                */
#define UART_IBRD   0x024               /* integer baud divisor                 */
#define UART_FBRD   0x028               /* fractional baud divisor (1/64ths)    */
#define UART_LCR_H  0x02C               /* line control                         */
#define UART_CR     0x030               /* control                              */
#define UART_IMSC   0x038               /* interrupt mask set/clear             */
#define UART_ICR    0x044               /* interrupt clear                      */
#define FR_RXFE     (1u << 4)           /* receive FIFO empty                   */
#define FR_TXFF     (1u << 5)           /* transmit FIFO full                   */
#define INT_RX      (1u << 4)           /* receive interrupt                    */
#define INT_RT      (1u << 6)           /* receive timeout interrupt            */

static volatile uint32_t *uart;         /* Device-mapped base from the FDT */
#define REG(off) uart[(off) / 4]

void pl011_init(volatile uint32_t *base, uint32_t clock_hz)
{
```

```
    uart = base;
    REG(UART_CR) = 0;                                   /* disable while configuring */
    uint32_t div = (4 * clock_hz + 115200 / 2) / 115200; /* 64 * clock / (16 * baud)   */
    REG(UART_IBRD) = div >> 6;
    REG(UART_FBRD) = div & 0x3F;
    REG(UART_LCR_H) = (3u << 5) | (1u << 4);            /* 8N1, FIFOs on; latches the divisors */
    REG(UART_IMSC) = INT_RX | INT_RT;
    REG(UART_CR) = (1u << 0) | (1u << 8) | (1u << 9);   /* UARTEN, TXE, RXE */
}

void pl011_putc(char c)
{
    while (REG(UART_FR) & FR_TXFF)
        ;
    REG(UART_DR) = (uint8_t)c;
}

void pl011_irq(struct trapframe *tf)
{
    (void)tf;
    while (!(REG(UART_FR) & FR_RXFE))
        tty_input_char(REG(UART_DR) & 0xFF);            /* Task 16.8: into the tty */
    REG(UART_ICR) = INT_RX | INT_RT;
}
```

With the FIFO on, the receive interrupt fires when the FIFO reaches its trigger level, and the receive-timeout interrupt catches the last few characters of a burst; enable both. Until Task 18.5 you can poll `FR_RXFE`.

Verify: `kmain` prints “Merhaba from AArch64, EL1” through the PL011, and `kprintf` works with `%lx` for 64-bit values.

Task 18.4 Exception vectors

Write `vectors.S` with one macro per 128-byte slot. Each slot makes room for a trap frame, saves `x0` and `x1`, puts its own number (0–15) in `x0` and branches to shared code that saves everything else and calls C. `trap_return` undoes it and ends in `eret`. In `arch_init`, load `VBAR_EL1` with the table’s address and execute `isb`.

kernel/arch/aarch64/vectors.S

```
#define TF_SIZE 288                     // sizeof(struct trapframe), a multiple of 16

.macro ventry kind                      // one 128-byte slot: save x0, x1, then share the rest
    .balign 0x80
    sub   sp, sp, #TF_SIZE
    stp   x0, x1, [sp, #16 * 0]
    mov   x0, #\kind                    // 0-3: EL1 with SP_EL0, 4-7: EL1 with SP_EL1,
    b     trap_common                   // 8-11: EL0 AArch64, 12-15: EL0 AArch32
.endm

    .text
    .balign 0x800                       // VBAR_EL1 needs 2 KiB alignment
    .global vectors
vectors:                                // in each group: sync, IRQ, FIQ, SError
    .irp kind, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
    ventry \kind
    .endr

trap_common:
    stp x2, x3, [sp, #16 * 1];    stp x4, x5, [sp, #16 * 2];    stp x6, x7, [sp, #16 * 3]
    stp x8, x9, [sp, #16 * 4];    stp x10, x11, [sp, #16 * 5];  stp x12, x13, [sp, #16 * 6]
```

```
    stp x14, x15, [sp, #16 * 7];  stp x16, x17, [sp, #16 * 8];  stp x18, x19, [sp, #16 * 9]
    stp x20, x21, [sp, #16 * 10]; stp x22, x23, [sp, #16 * 11]; stp x24, x25, [sp, #16 * 12]
    stp x26, x27, [sp, #16 * 13]; stp x28, x29, [sp, #16 * 14]
    mrs   x1, sp_el0
    stp   x30, x1, [sp, #16 * 15]       // x30 and the user's stack pointer
    mrs   x1, elr_el1
    mrs   x2, spsr_el1
    stp   x1, x2, [sp, #16 * 16]
    mrs   x1, esr_el1
    mrs   x2, far_el1
    stp   x1, x2, [sp, #16 * 17]
    mov   x1, x0                        // trap_handler(tf, kind)
    mov   x0, sp
    bl    trap_handler

    .global trap_return
trap_return:
    msr   daifset, #2                   // no IRQ between restoring ELR/SPSR and eret
    ldp   x1, x2, [sp, #16 * 16]
    msr   elr_el1, x1
    msr   spsr_el1, x2
    ldp   x30, x1, [sp, #16 * 15]
    msr   sp_el0, x1
    ldp x0, x1, [sp, #16 * 0];    ldp x2, x3, [sp, #16 * 1];    ldp x4, x5, [sp, #16 * 2]
    ldp x6, x7, [sp, #16 * 3];    ldp x8, x9, [sp, #16 * 4];    ldp x10, x11, [sp, #16 * 5]
    ldp x12, x13, [sp, #16 * 6];  ldp x14, x15, [sp, #16 * 7];  ldp x16, x17, [sp, #16 * 8]
    ldp x18, x19, [sp, #16 * 9];  ldp x20, x21, [sp, #16 * 10]; ldp x22, x23, [sp, #16 * 11]
    ldp x24, x25, [sp, #16 * 12]; ldp x26, x27, [sp, #16 * 13]; ldp x28, x29, [sp, #16 * 14]
    add   sp, sp, #TF_SIZE              // sp is the top of this task's kernel stack again
    eret
```

The trap frame matches that layout field for field, and the accessors of Task 16.5 hide it from generic code. The system call number is in `x8` and the first argument in `x0`, which is also where the result goes, so the generic dispatcher must read all its arguments before it calls `tf_set_ret`.

kernel/arch/aarch64/include/arch/trapframe.h

```
struct trapframe {
    uint64_t x[31];            /* x0..x30                    offset   0 */
    uint64_t sp_el0;           /* the user's stack pointer           248 */
    uint64_t elr;              /* ELR_EL1: where eret will go        256 */
    uint64_t spsr;             /* SPSR_EL1: the interrupted PSTATE   264 */
    uint64_t esr;              /* ESR_EL1: why the exception came    272 */
    uint64_t far;              /* FAR_EL1: the faulting address      280 */
};
_Static_assert(sizeof(struct trapframe) == 288, "vectors.S uses TF_SIZE = 288");

static inline unsigned long tf_sysno(const struct trapframe *tf)         { return tf->x[8]; }
static inline unsigned long tf_arg(const struct trapframe *tf, int n)    { return tf->x[n]; }
static inline void tf_set_ret(struct trapframe *tf, unsigned long v)     { tf->x[0] = v; }
static inline uintptr_t tf_pc(const struct trapframe *tf)                { return tf->elr; }
static inline void tf_set_pc(struct trapframe *tf, uintptr_t pc)         { tf->elr = pc; }
static inline uintptr_t tf_sp(const struct trapframe *tf)                { return tf->sp_el0; }
static inline bool tf_from_user(const struct trapframe *tf) { return (tf->spsr & 0xF) == 0; } /* EL0t */
```

The C handler decodes the slot number and, for synchronous exceptions, the exception class of Table 2. Give every class a name for the crash report, as you did for scause on RISC-V. Note the two different return addresses: after `svc`, `ELR_EL1` already points to the next instruction (unlike RISC-V, where you added 4 to `sepc`); after `brk` and after a fault it points at the instruction itself.

kernel/arch/aarch64/trap.c

```
enum { EC_UNKNOWN = 0x00, EC_FP_SIMD = 0x07, EC_SVC64 = 0x15, EC_IABT_LOW = 0x20,
       EC_IABT_CUR = 0x21, EC_DABT_LOW = 0x24, EC_DABT_CUR = 0x25, EC_SP_ALIGN = 0x26,
       EC_BRK64 = 0x3C };

void trap_handler(struct trapframe *tf, unsigned long kind)
{
    if (kind < 4 || kind >= 12)                    /* SP_EL0 at EL1, or AArch32 code */
        fatal_trap(tf, "unexpected vector");
    switch (kind & 3) {                            /* 0 sync, 1 IRQ, 2 FIQ, 3 SError */
    case 1:
        gic_handle_irq(tf);
        break;
    case 0: {
        unsigned ec = (tf->esr >> 26) & 0x3F;      /* exception class, ESR bits 31:26 */
        switch (ec) {
        case EC_SVC64:                             /* ELR already points past the svc */
            syscall_dispatch(tf);
            break;
        case EC_DABT_LOW: case EC_DABT_CUR: case EC_IABT_LOW: case EC_IABT_CUR:
            handle_page_fault(tf);                 /* generic; asks arch_fault_info() */
            break;
        case EC_BRK64:                             /* ELR points AT the brk: skip it */
            kprintf("brk #%lu at %lx\n", (unsigned long)(tf->esr & 0xFFFF), tf->elr);
            tf->elr += 4;
            break;
        default:
            fatal_trap(tf, ec_name(ec));           /* "SVE/SIMD/FP access", ... */
        }
        break;
    }
    default:
        fatal_trap(tf, "FIQ or SError");
    }
}
```

Verify: A test that executes `brk ##7` (in inline assembly) prints its immediate and continues. A load from an unmapped address in a test shows "data abort from EL1" with the address from `FAR_EL1` and the register dump.

Task 18.5 Interrupts and time

Find the GIC in the devicetree (`arm,cortex-a15-gic`, two `reg` ranges), configure the distributor, enable it and the CPU interface, and set the priority mask. Then route device interrupts by INTID through Phase 3's handler table, grown to the number of INTIDs the GIC reports. Phase 8's lesson still holds: write `GICC_EOIR` before you call a handler that may switch tasks. Until an interrupt is ended, the CPU interface blocks every interrupt of the same or lower priority, and preemption would stop.

kernel/arch/aarch64/gic.c

```
#define GICD_CTLR       0x000          /* distributor: on/off                    */
#define GICD_TYPER      0x004          /* bits 4:0 = number of INTIDs / 32 - 1   */
#define GICD_ISENABLER  0x100          /* one bit per INTID: write 1 to enable   */
#define GICD_ICENABLER  0x180          /* write 1 to disable                     */
#define GICD_IPRIORITYR 0x400          /* one byte per INTID, lower = more urgent */
#define GICD_ITARGETSR  0x800          /* one byte per INTID: which CPUs (SPIs)  */
#define GICC_CTLR       0x000          /* CPU interface: on/off                  */
#define GICC_PMR        0x004          /* priority mask: only priorities below it */
#define GICC_IAR        0x00C          /* read = acknowledge, gives the INTID    */
#define GICC_EOIR       0x010          /* write = end of interrupt               */
```

```
static volatile uint8_t *gicd, *gicc;   /* Device-mapped, bases from the FDT */
#define D32(off) (*(volatile uint32_t *)(gicd + (off)))
#define C32(off) (*(volatile uint32_t *)(gicc + (off)))

void gic_init(void)
{
    unsigned nr = 32 * ((D32(GICD_TYPER) & 0x1F) + 1);
    D32(GICD_CTLR) = 0;                             /* off while configuring */
    for (unsigned id = 0; id < nr; id++) {
        gicd[GICD_IPRIORITYR + id] = 0xA0;          /* these registers are byte-accessible */
        if (id >= 32)
            gicd[GICD_ITARGETSR + id] = 0x01;       /* every SPI goes to CPU interface 0 */
    }
    D32(GICD_CTLR) = 1;
    C32(GICC_PMR) = 0xF0;                           /* 0 would mask everything */
    C32(GICC_CTLR) = 1;
}

void arch_irq_unmask(unsigned irq) { D32(GICD_ISENABLER + 4 * (irq / 32)) = 1u << (irq % 32); }
void arch_irq_mask(unsigned irq)   { D32(GICD_ICENABLER + 4 * (irq / 32)) = 1u << (irq % 32); }

void gic_handle_irq(struct trapframe *tf)
{
    uint32_t iar = C32(GICC_IAR);                   /* acknowledge */
    unsigned id = iar & 0x3FF;
    if (id == 1023)                                 /* spurious: nothing to end */
        return;
    C32(GICC_EOIR) = iar;                           /* before a handler that may switch tasks */
    irq_dispatch(id, tf);                           /* the handler table, indexed by INTID */
}
```

The timer reloads its down-counter on every tick. Its PPI is level-triggered: until you write `CNTV_TVAL_EL0` again, the interrupt stays asserted.

kernel/arch/aarch64/timer.c

```
#define TIMER_INTID 27                  /* EL1 virtual timer, a PPI */
static uint64_t ticks_per_irq;

static void timer_irq(struct trapframe *tf)
{
    __asm__ volatile ("msr cntv_tval_el0, %0" : : "r"(ticks_per_irq));  /* next tick */
    sched_tick(tf);                     /* Phase 8: accounting, sleepers, preemption */
}

void arch_timer_init(unsigned hz)
{
    uint64_t freq;
    __asm__ volatile ("mrs %0, cntfrq_el0" : "=r"(freq));     /* never hard-code it */
    ticks_per_irq = freq / hz;
    register_interrupt_handler(TIMER_INTID, timer_irq);
    __asm__ volatile ("msr cntv_tval_el0, %0" : : "r"(ticks_per_irq));
    __asm__ volatile ("msr cntv_ctl_el0, %0" : : "r"(1UL));   /* ENABLE = 1, IMASK = 0 */
    arch_irq_unmask(TIMER_INTID);
}

uint64_t arch_cycles(void)
{
    uint64_t t;
    __asm__ volatile ("isb; mrs %0, cntvct_el0" : "=r"(t));   /* isb: don't read it early */
    return t;
}
```

Finally take the PL011's INTID from its `interrupts` property, register `pl011_irq` and unmask it.

Verify: `uptime` in the kernel monitor advances by 100 ticks per second of wall-clock time, keys typed on the serial console reach the monitor through the receive interrupt, and a counter per INTID in the monitor shows 27 and 33 rising.

Task 18.6 The MMU, carefully

Turn the MMU on in `boot.S`, before any C code runs, with one static level-1 table of two 1 GiB blocks: Device memory for the first gigabyte (the GIC, the PL011, virtio) and Normal memory for the second (RAM, with the kernel). Because `KERNEL_BASE` has zeros in bits 38:30, the same table works for both bases: through `TTBR0_EL1` it is an identity map, needed for the few instructions between turning the MMU on and jumping high; through `TTBR1_EL1` it is the high half. Now set `KERNEL_BASE` back to its real value and drop `-mstrict-align`. Insert this code just before `bl kmain`.

kernel/arch/aarch64/boot.S (MMU)

```
#define BLOCK      (1 << 0)              // valid level-1 block: 1 GiB
#define ATTR(i)    ((i) << 2)            // AttrIndx: 0 = Device, 1 = Normal (MAIR_EL1)
#define SH_INNER   (3 << 8)
#define AF         (1 << 10)
#define PXN        (1 << 53)
#define UXN        (1 << 54)
#define DEV_BLOCK  (BLOCK | ATTR(0) | AF | PXN | UXN)
#define MEM_BLOCK  (BLOCK | ATTR(1) | SH_INNER | AF | UXN)
#define MAIR_VALUE ((0x00 << 0) | (0xff << 8))   // Attr0 Device-nGnRnE, Attr1 Normal WB
#define TCR_VALUE  ((25 << 0) | (1 << 8) | (1 << 10) | (3 << 12) | (0 << 14) | \
                    (25 << 16) | (1 << 24) | (1 << 26) | (3 << 28) | (2 << 30))

    .section .data
    .balign 4096
boot_l1:                                  // one level-1 table, used by TTBR0 and TTBR1
    .quad 0x00000000 | DEV_BLOCK          // PA 0-1 GiB: GIC, PL011, virtio-mmio
    .quad 0x40000000 | MEM_BLOCK          // PA 1-2 GiB: RAM, with the kernel in it
    .fill 510, 8, 0

    .text
enable_mmu:
    adrp  x1, boot_l1
    msr   ttbr0_el1, x1             // identity map: VA = PA for the next few instructions
    msr   ttbr1_el1, x1             // high half: VA = KERNEL_BASE + PA
    ldr   x1, =MAIR_VALUE
    msr   mair_el1, x1
    ldr   x1, =TCR_VALUE
    mrs   x2, id_aa64mmfr0_el1
    and   x2, x2, #0xf              // PARange: how many physical address bits exist
    bfi   x1, x2, #32, #3           // TCR_EL1.IPS (bits 34:32) = PARange
    msr   tcr_el1, x1
    isb
    tlbi  vmalle1                   // forget anything cached before reset
    dsb   nsh
    mrs   x1, sctlr_el1
    orr   x1, x1, #(1 << 0)         // M: MMU on
    orr   x1, x1, #(1 << 2)         // C: data cache on
    orr   x1, x1, #(1 << 12)        // I: instruction cache on
    msr   sctlr_el1, x1
    isb                             // from here on every fetch is translated
    ldr   x1, =high_entry           // the link-time address, in the TTBR1 half
    br    x1
high_entry:
    ldr   x1, =boot_stack_top       // move sp to its high-half alias as well
    mov   sp, x1
    bl    kmain
```

The `TCR_EL1` value spells out the 39-bit layout: `T0SZ` and `T1SZ` (bits 5:0 and 21:16) are 25; `IRGN`, `ORGN` and `SH` make table walks cacheable and inner shareable; and the two granule fields are encoded differently, `TG0 = 0b00` but `TG1 = 0b10` for 4 KiB. `IPS` is copied from `ID_AA64MMFR0_EL1.PARange`, so the same code runs on any core.

The boot table maps far too much. In C, build the real kernel tables: the direct map with 2 MiB blocks covering only the RAM listed in `/memory`, the kernel image with 4 KiB pages so that `.text` is read-only and `.rodata` not executable (Phase 15's hardening), and each device the FDT names as a Device page at `P2V(base)`. Keep `PXN` and `UXN` on every direct-map block and every data page, so the only kernel pages EL1 may execute are those of `.text`, and no page is both writable and executable. Load them into `TTBR1_EL1`, flush the whole TLB, and give `TTBR0_EL1` an empty table for kernel threads. The header of the new memory layout:

kernel/arch/aarch64/include/arch/memlayout.h

```
#define KERNEL_BASE     0xFFFFFF8000000000UL    /* start of the TTBR1 half */
#define USER_BASE       0x00400000UL
#define USER_TOP        0x0000008000000000UL    /* 2^39: end of the TTBR0 half */
#define USER_STACK_TOP  USER_TOP
#define PAGE_SIZE       4096UL
#define P2V(pa)         ((void *)((uintptr_t)(pa) + KERNEL_BASE))
#define V2P(va)         ((uintptr_t)(va) - KERNEL_BASE)
```

The descriptor bits go into `<arch/mmu.h>`. `arch_mmu_map` walks the three levels exactly like Phase 17's two-level walk, with one more step: index bits 38:30, then 29:21, then 20:12, and a missing table becomes a zeroed page linked in as `V2P(page) | PTE_VALID | PTE_TABLE`. After writing the leaf it executes `dsb ishst`, so the table walker sees the entry. Translating the generic `VM_*` flags is where the inverted bits of Table 3 bite:

kernel/arch/aarch64/include/arch/mmu.h

```
typedef uint64_t pte_t;
#define PTE_VALID    (1UL << 0)
#define PTE_TABLE    (1UL << 1)         /* with VALID: a table (levels 1-2) or a page (level 3) */
#define PTE_ATTR(i)  ((pte_t)(i) << 2)  /* AttrIndx into MAIR_EL1 */
#define MT_DEVICE    0                  /* Attr0 = 0x00: Device-nGnRnE */
#define MT_NORMAL    1                  /* Attr1 = 0xFF: Normal, write-back cacheable */
#define PTE_EL0      (1UL << 6)         /* AP[1]: EL0 may access */
#define PTE_RDONLY   (1UL << 7)         /* AP[2]: read-only at every level */
#define PTE_SH_INNER (3UL << 8)
#define PTE_AF       (1UL << 10)        /* access flag: 0 faults on first use */
#define PTE_NG       (1UL << 11)        /* not global: belongs to one address space */
#define PTE_PXN      (1UL << 53)        /* EL1 may not execute */
#define PTE_UXN      (1UL << 54)        /* EL0 may not execute */
#define PTE_COW      (1UL << 55)        /* software bit (55-58 are ignored by the MMU) */
#define PTE_ADDR(e)  ((e) & 0x0000FFFFFFFFF000UL)
```

kernel/arch/aarch64/mmu.c

```
static pte_t vm_to_pte(unsigned flags)
{
    pte_t e = PTE_VALID | PTE_TABLE | PTE_AF | PTE_SH_INNER | PTE_ATTR(MT_NORMAL);
    if (!(flags & VM_WRITE) || (flags & VM_COW))
        e |= PTE_RDONLY;
    if (flags & VM_COW)
        e |= PTE_COW;
    if (flags & VM_USER) {
        e |= PTE_EL0 | PTE_NG | PTE_PXN;            /* the kernel never runs user code */
        if (!(flags & VM_EXEC))
            e |= PTE_UXN;
    } else {
```

```
        e |= PTE_UXN;
        if (!(flags & VM_EXEC))
            e |= PTE_PXN;
    }
    return e;
}

void arch_tlb_flush(uintptr_t va)
{
    uint64_t op = (va >> 12) & ((1UL << 44) - 1);   /* VA[55:12] in bits 43:0, ASID 0 */
    __asm__ volatile ("dsb ishst\n\t"       /* finish the table write first   */
                      "tlbi vae1is, %0\n\t" /* drop that page from every TLB  */
                      "dsb ish\n\t"         /* wait until the TLBs are clean  */
                      "isb"                 /* and refetch with the new view  */
                      : : "r"(op) : "memory");
}
```

A new mapping needs no TLB flush, because the TLBs never hold entries that fault; `arch_mmu_protect` and `arch_mmu_unmap` change live entries and must call `arch_tlb_flush`. `arch_mmu_switch` writes `TTBR0_EL1`, executes `isb`, and, since Turk-OS uses no ASIDs yet, flushes the TLB with `tlbi vmalle1`, `dsb ish` and `isb`. Two more differences from x86 and RISC-V: Armv8.0 has no equivalent of `sstatus.SUM`, so with `arch_uaccess_begin` and `arch_uaccess_end` empty, `copy_from_user` reads user pages directly, and its range check now ends at `USER_TOP`.

Armv8.1 adds **PAN** (Privileged Access Never), the ARM counterpart of SMAP and of a clear SUM: while `PSTATE.PAN` is 1, an EL1 load or store to an EL0-accessible page faults. It is optional, and worth the few lines when the CPU has it (`ID_AA64MMFR1_EL1`, bits 23:20, non-zero; QEMU's `-cpu max` does, `cortex-a72` doesn't). Clear `SCTLR_EL1.SPAN` (bit 23), so every exception taken to EL1 sets PAN, and set it once at boot. Then choose one of two ways for uaccess. Either the hooks toggle PAN, `msr s3_0_c4_c2_3, xzr` (the PAN register by its generic name, so the assembler needs no `-march=armv8.1-a`) before the copy and writing `1 << 22` back after it, or the copy loops use `LDTR` and `STTR`, the unprivileged loads and stores. PAN doesn't block those, because they are checked with EL0's permissions, so a kernel address passed as a user pointer faults even if the range check missed it. Either way, any other EL1 access to user memory becomes a permission fault.

Finally fill in the fault description for the generic handler:

kernel/arch/aarch64/trap.c (page faults)

```
void arch_fault_info(const struct trapframe *tf, struct fault_info *fi)
{
    unsigned ec  = (tf->esr >> 26) & 0x3F;
    unsigned fsc = tf->esr & 0x3F;                  /* DFSC or IFSC: type and level */
    fi->addr    = tf->far;                          /* the faulting virtual address */
    fi->exec    = (ec == 0x20 || ec == 0x21);       /* instruction abort            */
    fi->user    = (ec == 0x20 || ec == 0x24);       /* taken from EL0               */
    fi->write   = !fi->exec && (tf->esr & (1u << 6));   /* WnR, data aborts only    */
    fi->present = (fsc & 0x3C) != 0x04;             /* 0x04-0x07: translation fault */
}
```

Verify: The kernel prints from its high-half address (`0xFFFFFF80402...` in a backtrace); the Phase 6 fault tests pass: a write to `.text` is a permission fault at level 3 (fault status `0x0F`), an unmapped address a translation fault (`0x04`–`0x07`, by the level where the walk stopped); and your own `pagetable` monitor command shows Device attributes only on device pages and `PXN` clear only on `.text`. With `-cpu max` and PAN on, a kernel read of a user pointer outside uaccess is a permission fault.

Task 18.7 Sixty-four bits everywhere

Task 16.9 made the generic code compile cleanly for aarch64, but compiling is not running. This is the first time Turk-OS executes with 64-bit pointers, and some bugs only show at run time. Fix them in generic code, not in `arch/`, and run the fixed code on i386 and riscv32 too. The bugs this port typically finds:

Bug	What happens and the fix
pointer stored in `uint32_t`	kernel addresses lose their top half; use `uintptr_t`, and treat any pointer-to-int cast warning as an error
`addr & ~0xFFFu`	`~0xFFFu` is a 32-bit `0xFFFFF000`, so the mask also clears bits 63:32; use `~(uintptr_t)(PAGE_SIZE - 1)`
Phase 10's `syscall_fn` with `uint32_t` arguments	user pointers lose their top bits (the user stack sits just below 2^{39}), and a result kept in a `uint32_t` reaches user mode as `0x00000000FFFFFFFE` instead of -2; take `unsigned long` arguments and return `long`
`%x` for addresses and sizes	half the value is printed; use `%lx`, `%zu`, and let `-Wformat` find them
the initial user stack	`argv` and `envp` slots are 8 bytes, and `sp` must be 16-byte aligned at `_start`
the ELF loader	`EM_386` and `EM_RISCV` images are 32-bit; AArch64 programs are ELF64

The ELF loader learns the 64-bit structures and accepts `ELFCLASS64` (2 in `e_ident[4]`) with `EM_AARCH64` (183), and only those, on this port. `Elf64_Ehdr` is 64 bytes and `Elf64_Phdr` 56: entry, offsets, addresses and sizes grow to 64 bits, and in the program header `p_flags` moves up to sit right after `p_type`. Check both sizes with `_Static_assert`. `paddr_t` was already 64 bits since Task 16.7; AArch64 is where that matters.

Verify: The evil programs, rebuilt for AArch64, behave as listed in Task 10.9 and as on RISC-V: `ro_read` returns `-EFAULT`, `lazy_read` succeeds, `cow_read`'s data is seen only by the reader, and `nxdata` dies of an instruction abort from EL0 (`UXN`). One more case is new: a `write` whose buffer lies at `0x0000008000000000` (just past `USER_TOP`) returns `-EFAULT`.

Task 18.8 Tasks and locks

`arch_switch` saves only what the AAPCS64 makes callee-saved: `x19`–`x28`, the frame pointer and the link register, 96 bytes, and the stack pointer itself goes into `*save`. A new task's first context points `x30` at a trampoline that enables interrupts and calls the entry function.

kernel/arch/aarch64/switch.S

```
// void arch_switch(struct context **save, struct context *load)
// struct context = x19..x28, x29 (frame pointer), x30 (return address): 96 bytes.
    .text
    .global arch_switch
arch_switch:
    sub   sp, sp, #96               // keeps sp 16-byte aligned
    stp   x19, x20, [sp, #0];   stp   x21, x22, [sp, #16]
    stp   x23, x24, [sp, #32];  stp   x25, x26, [sp, #48]
    stp   x27, x28, [sp, #64];  stp   x29, x30, [sp, #80]
    mov   x2, sp                    // str cannot store sp directly
    str   x2, [x0]                  // *save = this task's context
    mov   sp, x1                    // switch to the other task's kernel stack
    ldp   x19, x20, [sp, #0];   ldp   x21, x22, [sp, #16]
    ldp   x23, x24, [sp, #32];  ldp   x25, x26, [sp, #48]
    ldp   x27, x28, [sp, #64];  ldp   x29, x30, [sp, #80]
    add   sp, sp, #96
    ret                             // to wherever that task called arch_switch

    .global task_trampoline         // arch_task_init(): x19 = entry, x20 = arg
task_trampoline:
```

```
    msr   daifclr, #2              // new tasks start with IRQs enabled
    mov   x0, x20
    blr   x19                      // entry(arg)
    bl    task_exit                // never returns

    .global arch_return_to_user    // _Noreturn void arch_return_to_user(struct trapframe *tf)
arch_return_to_user:
    mov   sp, x0                   // the frame sits at the top of the kernel stack
    b     trap_return
```

kernel/arch/aarch64/proc.c

```
void arch_task_init(struct task *t, void (*entry)(void *), void *arg)
{
    struct context *c = (struct context *)((char *)t->kstack + KSTACK_SIZE) - 1;
    memset(c, 0, sizeof *c);
    c->x19 = (uint64_t)entry;              /* task_trampoline calls x19(x20) */
    c->x20 = (uint64_t)arg;
    c->x30 = (uint64_t)task_trampoline;    /* arch_switch's ret lands here   */
    t->context = c;
}
```

Interrupt control uses `DAIF`. `msr daifset, #2` sets only the I bit (the immediate's bits are D, A, I, F from 8 down to 1), and restoring writes the whole saved `DAIF` back, so nesting works exactly as in Task 9.1. The spinlock's atomic exchange becomes a pair of exclusive instructions: `ldaxr` loads and arms the exclusive monitor, `stlxr` stores only if nobody wrote the location in between and reports failure otherwise, and the loop retries. The "a" and "l" give the acquire and release ordering that `__atomic_exchange_n` asked for in Phase 9. Exclusives are only guaranteed on Normal, cacheable memory, which is one more reason why the MMU is on before the first lock is taken.

kernel/arch/aarch64/include/arch/irq.h

```
static inline unsigned long irq_save(void)
{
    unsigned long flags;
    __asm__ volatile ("mrs %0, daif\n\tmsr daifset, #2" : "=r"(flags) : : "memory");
    return flags;                                  /* DAIF.I (bit 7) as it was */
}

static inline void irq_restore(unsigned long flags)
{
    __asm__ volatile ("msr daif, %0" : : "r"(flags) : "memory");
}

static inline void irq_enable(void)  { __asm__ volatile ("msr daifclr, #2" : : : "memory"); }
static inline void irq_disable(void) { __asm__ volatile ("msr daifset, #2" : : : "memory"); }
static inline void cpu_idle(void)    { __asm__ volatile ("wfi"); }
```

kernel/arch/aarch64/include/arch/atomic.h

```
#define ARCH_HAS_ATOMICS 1

static inline uint32_t atomic_xchg(volatile uint32_t *p, uint32_t v)
{
    uint32_t old, failed;
    __asm__ volatile ("1: ldaxr %w0, [%2]\n\t"      /* load and open the exclusive monitor */
                      "   stlxr %w1, %w3, [%2]\n\t" /* store only if nobody wrote since   */
                      "   cbnz  %w1, 1b"            /* lost the reservation: try again    */
                      : "=&r"(old), "=&r"(failed) : "r"(p), "r"(v) : "memory");
    return old;
}
```

Verify: The Phase 8 and Phase 9 suites pass: three preempted threads share the CPU, 1000 create and exit cycles leak nothing, and the producer–consumer and dining-philosophers tests finish. A disassembly of `spin_lock` shows `ldaxr` and `stlxr`.

Task 18.9 Into EL0 and back

`arch_user_entry` fills the trap frame that `arch_return_to_user` will `eret` from. `SPSR_EL1` = 0 means mode EL0t with no interrupt class masked; `ELR_EL1` is the program's entry; `SP_EL0` is its stack. The trap frame sits at the top of the task's kernel stack, so after `eret`, `SP_EL1` points exactly there, ready for the next exception: nothing like Phase 10's `tss_set_kernel_stack` is needed.

kernel/arch/aarch64/proc.c (continued)

```
void arch_user_entry(struct trapframe *tf, uintptr_t entry, uintptr_t sp)
{
    memset(tf, 0, sizeof *tf);              /* no kernel values leak to user mode */
    tf->elr = entry;                        /* eret jumps here                    */
    tf->sp_el0 = sp & ~(uintptr_t)0xF;      /* AAPCS64: 16-byte aligned           */
    tf->spsr = 0;                           /* M = EL0t, D A I F clear: IRQs on   */
}
```

System calls arrive as EC `0x15` at offset `0x400` and go through the same generic dispatcher as on x86 and RISC-V, with the same Linux i386 numbers: `write` is still 4, whatever Linux's own arm64 table says. `arch_signal_frame` copies the trap frame to the user stack, sets `ELR_EL1` to the handler and `x0` to the signal number, and puts the restorer in `x30`: on AArch64 the handler's `ret` goes to the link register, so nothing needs to be pushed. `arch_sigreturn` copies the saved frame back after checking that its `SPSR` still says EL0t, or a program could forge a frame that returns to EL1.

Verify: A hand-made user program loops on `svc getpid`; `info registers` while it runs shows `EL0t`, and a forged `sigreturn` frame with `SPSR` = `0x5` kills the process instead of entering the kernel.

Task 18.10 virtio again, and the whole userland

`kernel/drivers/virtio_blk.c` and `virtio_mmio.c` from Task 17.10 compile unchanged; only the base and the INTID come from the FDT, and the INTID goes to `arch_irq_unmask` instead of the PLIC. Probe all 32 slots, skipping those whose device ID is 0: a comment in QEMU's `hw/arm/virt.c` explains that `-device` options are assigned from the highest address down, so a single disk usually sits in the last slot. Keep `-global virtio-mmio.force-legacy=false`, or QEMU offers the legacy interface (version 1). The queue address registers come in low and high halves; on RV32 the high halves were always 0, now they carry real bits. Before the `QueueNotify` write the shared driver needs a barrier that orders the descriptor writes before it: `fence` on RISC-V, `dsb st` here.

The userland needs a start file, the system call stubs, `user/arch/aarch64/user.ld` (programs at `0x00400000`, as everywhere) and the same `-mgeneral-regs-only`. Then rebuild Turk-libc, `tsh` and the utilities.

user/arch/aarch64/crt0.S

```
// user/arch/aarch64/crt0.S -- the kernel enters here with sp -> argc, argv[], envp[]
    .text
    .global _start
_start:
    ldr   x0, [sp]                  // argc
    add   x1, sp, #8                // argv: 8-byte pointers in LP64
    add   x2, x1, x0, lsl #3        // skip argv[0..argc-1]
    add   x2, x2, #8                // and its NULL: envp
    bl    main
    bl    exit                      // exit(main's return value), never returns
```

user/arch/aarch64/syscall.c

```
static inline long syscall3(long n, long a, long b, long c)
{
    register long x8 __asm__("x8") = n;     /* the Linux i386 number, as on x86 */
    register long x0 __asm__("x0") = a;
    register long x1 __asm__("x1") = b;
    register long x2 __asm__("x2") = c;
    __asm__ volatile ("svc #0" : "+r"(x0) : "r"(x8), "r"(x1), "r"(x2) : "memory");
    return x0;                              /* result, or -errno */
}

long write(int fd, const void *buf, unsigned long len) { return syscall3(4, fd, (long)buf, len); }
int  getpid(void)                                      { return syscall3(20, 0, 0, 0); }
void exit(int code)                                    { syscall3(1, code, 0, 0); for (;;) { } }
```

For `make test`, the board has no `isa-debug-exit` or `sifive_test` device. Instead QEMU implements Arm **semihosting**, a debugger interface: with `-semihosting-config enable=on,target=native`, the instruction `hlt #0xf000` at EL1 becomes a request to QEMU, with the operation in `w0` and a parameter pointer in `x1`. `SYS_EXIT` (`0x18`) on AArch64 takes a two-word block; when its first word is `ADP_Stopped_ApplicationExit` (`0x20026`), QEMU exits with the second word as its exit status. User code cannot do this unless you also pass `userspace=on`, which you should not.

kernel/arch/aarch64/semihost.c

```
_Noreturn void arch_test_exit(int failed)
{
    static uint64_t block[2];
    block[0] = 0x20026;                         /* ADP_Stopped_ApplicationExit */
    block[1] = failed ? 1 : 0;                  /* QEMU's exit status          */
    register uint64_t op  __asm__("x0") = 0x18; /* SYS_EXIT                    */
    register uint64_t arg __asm__("x1") = (uint64_t)block;
    __asm__ volatile ("hlt #0xf000" : : "r"(op), "r"(arg) : "memory");
    for (;;)                                    /* only without -semihosting   */
        __asm__ volatile ("wfi");
}
```

Verify: `make ARCH=aarch64 run` boots to `turk@os:/$` with the root on virtio-blk; `make ARCH=aarch64 test` prints every result and `ALL TESTS PASSED`, and breaking one test makes QEMU exit with status 1. The Phase 15 fuzzer runs for an hour without a kernel panic, and the CI matrix of Task 16.11, now with `aarch64`, runs the evil programs and a short fuzzer run green on all three architectures.

Task 18.11 Turk-OS 2.0

Now bring the three architectures together. TurkFS has no endianness or word-size problem: all three targets are little-endian and the on-disk format uses fixed-width fields, so one image must work everywhere. Prove it: create an image with `mkfs.minix -3`, write files on AArch64, check it with `fsck.minix -f`, boot the same file on riscv32 (virtio) and on i386 (ATA, `-drive if=ide`), and write from there too.

terminal

```
make release-all                                  # i386 ISO, riscv32 and aarch64 kernels, one TurkFS image
make ARCH=aarch64 run DISK=build/turkfs-2.0.img   # write /home/turk/from-arm
fsck.minix -f build/turkfs-2.0.img
make ARCH=riscv32 run DISK=build/turkfs-2.0.img   # read it, write /home/turk/from-riscv
make ARCH=i386 run DISK=build/turkfs-2.0.img      # read both
```

Run the Phase 15 benchmarks on all three and fill a table like the one below. `arch_cycles` counts different things on each machine (`rdtsc`, `rdtime`, `cntvct_el0`), so convert every result to nanoseconds

with that counter's frequency, and remember that under QEMU you are comparing emulators, not processors. Explain the largest difference you find.

Benchmark (median, ns)	i386	riscv32	aarch64
null system call (`getpid`)			
context switch (two processes, two pipes)			
`fork` + `exit` + `wait`			
16 MiB file write, then read with a warm cache			
boot to the `tsh` prompt			

Then write: add a porting chapter to the Phase 15 design reflection (what leaked, what the interface looks like now, which port was hardest and why, what you would design differently from the start), update the architecture document with the AArch64 memory map and the `docs/porting.md` conclusions, and check that `make release-all` works from a clean clone. Tag the commit `v2.0`.

Verify: The image passes `fsck.minix -f` after each of the three boots, and every architecture reads the files the other two wrote. `make test` passes for `ARCH=i386`, `riscv32` and `aarch64` on the tagged commit.

Task 18.12 Optional: Raspberry Pi 4

The Pi 4's Cortex-A72 is the CPU you have been emulating, so the kernel is the same; the board around it is not. Write the image to the FAT boot partition of an SD card that already holds the Raspberry Pi firmware, next to this `config.txt`. Leave the SD card itself alone: an SD driver is out of scope, so the root is the initrd.

config.txt

```
arm_64bit=1
kernel=kernel8.img
kernel_address=0x200000
enable_uart=1
dtoverlay=disable-bt
initramfs initrd.tar followkernel
```

On the Pi 4 the PL011 normally serves Bluetooth and the weaker mini UART is on the header pins; the `disable-bt` overlay puts the PL011 on GPIO 14 (pin 8, transmit) and GPIO 15 (pin 10, receive). Connect a 3.3 V USB serial adapter there and to a ground pin; never a 5 V or RS-232 one. The firmware's stub starts core 0 at EL2 with `x0` pointing to the devicetree, which is why Task 18.2 already drops to EL1. Before you touch the card, boot QEMU with `-machine virt,virtualization=on`, which also starts at EL2.

	QEMU `virt`	Raspberry Pi 4 (from its devicetree)
entry	EL1 (EL2 with `virtualization=on`)	EL2
RAM, load address	from `0x40000000`; kernel at `0x40200000`	from 0; kernel at `0x200000`: a second link, `BOARD=rpi4`
boot table	Device 0–1 GiB, Normal 1–2 GiB	Normal from 0, Device for the gigabyte holding the peripherals
interrupt controller	GICv2 at `0x08000000`	GIC-400 (`arm,gic-400`), a GICv2, at `0xFF841000`
console	PL011 at `0x09000000`, INTID 33, 24 MHz	PL011 at `0xFE201000`, SPI 121 = INTID 153, clock from the tree
timer	virtual timer, INTID 27	the same
root	TurkFS on virtio-blk	the initrd only
other cores	powered off until PSCI	waiting in the firmware's spin loop

Honour the devicetree's `/memreserve/` entries and `/reserved-memory`: the firmware's stub and its spin table live at the bottom of RAM.

Verify: The serial terminal (115200 baud) shows "Merhaba from a Raspberry Pi 4, EL1", the memory map from the firmware's devicetree, and finally `turk@os:/$` running from the initrd.

5 Testing and debugging

Start QEMU with `-d int,guest_errors -D build/aarch64/qemu.log` to log every exception with its ESR, and use `gdb-multiarch build/aarch64/kernel.elf` with `target remote :1234`; `info registers` in the QEMU monitor shows the current level as `EL1h` or `EL0t`. When the machine stops dead right after the MMU is turned on, step through `enable_mmu` with `stepi` and check each system register with `p $tcr_el1`.

Symptom	Likely cause	What to do
Undefined instruction, EC `0x07`, in the first `memcpy` or `kprintf`	The compiler used SIMD registers and `CPACR_EL1.FPEN` traps them	Build the kernel and the userland with `-mgeneral-regs-only`; check `ARCH_CFLAGS` reaches every file.
No output at all; `x0` holds no devicetree	The image has no arm64 header (QEMU then loads it at `0x40080000`), or an ELF was passed	Check the magic `0x644d5241` at offset 56 of `kernel8.img`; pass the raw image to `-kernel`.
The machine hangs right after `SCTLR_EL1.M` is set	No `isb`, the code is not identity-mapped through `TTBR0_EL1`, or a wrong `TCR_EL1` (`TG1` must be `0b10`)	Step with GDB; compare `TCR_EL1` with the field list of Task 18.6.
Devices stop responding once the MMU is on	MMIO mapped as Normal memory, or not mapped	Map every device with AttrIndx 0 (Device-nGnRnE).
An access-flag fault on every page (fault status `0x09`–`0x0B`)	AF clear in the descriptors	Set `PTE_AF` in every block and page descriptor.
A spinlock spins for ever, or faults, early in boot	Exclusives on Device memory: the MMU or the caches are still off	Turn the MMU on in `boot.S`, before the first lock.
Random corruption after changing page tables	Missing `dsb ishst` before the `tlbi`, or `dsb ish` and `isb` after it	Use `arch_tlb_flush` for every live change.
An exception loop, or handlers run for the wrong exception	`VBAR_EL1` not 2 KiB aligned, or an entry longer than 0x80 bytes	Keep `.balign 0x800` and keep each slot at four instructions.
Kernel exceptions arrive at the `SP_EL0` entries	The kernel runs in EL1t: `SPSel` is 0	`msr spsel, #1` in `boot.S`; `SPSR_EL2` must say EL1h (`0x3c5`).
SP alignment fault, EC `0x26`	A trap frame or context size that is not a multiple of 16, or a misaligned user stack	Keep `TF_SIZE` 288 and the context 96; align `sp` in `arch_user_entry`.
`eret` to EL0 faults at once	`SPSR_EL1` mode bits not 0, `ELR_EL1` wrong, or the text mapped with `UXN`	Dump the frame before `eret`; check the PTE of the entry point.
GIC interrupts never arrive	Distributor or CPU interface not enabled, `GICC_PMR` still 0, the INTID not enabled, or `DAIF.I` set	Read back `GICD_CTLR`, `GICC_CTLR`, `GICC_PMR` and `GICD_ISENABLER`.
An interrupt storm from the timer	`CNTV_TVAL_EL0` not reloaded, so the level-triggered PPI stays asserted	Reload the timer first in its handler.
Pointers above 4 GiB are corrupted	A 64-bit truncation: `uint32_t` or a 32-bit mask	See Task 18.7; build with `-Werror` on the generic code.
`make test` ends with an undefined instruction at `arch_test_exit`	QEMU runs without semihosting, so `hlt` is undefined	Add `-semihosting-config enable=on,target=native` to the test flags.

Symptom	Likely cause	What to do
No output on the Raspberry Pi	PL011 not on the header pins, wrong UART clock, TX and RX swapped, or the EL2 path broken	Check `config.txt`; take the clock from the tree; test EL2 entry in QEMU first.

6 Milestone

Milestone: Turk-OS 2.0: three architectures

- ☐ `qemu-system-aarch64 -machine virt -cpu cortex-a72` boots to the `tsh` prompt, from EL1 and with `virtualization=on` from EL2.
- ☐ Exceptions are decoded from `ESR_EL1` with names in crash reports; `brk` returns; user faults become signals.
- ☐ GICv2, the virtual timer at 100 Hz from `CNTFRQ_EL0`, and PL011 receive interrupts work; the monitor runs over serial.
- ☐ The MMU runs with `TTBR1_EL1` for the kernel and `TTBR0_EL1` per process, Device and Normal memory, and copy-on-write.
- ☐ ELF64 programs run at EL0 with `svc` system calls and the Linux i386 numbers; the evil programs are contained.
- ☐ `PXN/UXN` keep every page writable or executable, never both: a write to kernel text and an execute from a user data page both fault. PAN is on when the CPU has it (optional).
- ☐ `make ARCH=aarch64 test` passes through semihosting, the evil programs and the fuzzer pass on AArch64 in CI, and the fuzzer runs for an hour without a panic.
- ☐ One TurkFS image is written and read on all three architectures and passes `fsck.minix -f`.
- ☐ `make release-all` works from a clean clone, the benchmark table and the porting chapter are written, and `v2.0` is tagged.

Self-check questions

1. x86 has one `CR3`; AArch64 has `TTBR0_EL1` and `TTBR1_EL1`. What does the second register save the kernel on every process creation and switch?
2. What does the access flag let an operating system do, and why does Turk-OS set it everywhere?
3. Why must device registers be mapped Device-nGnRnE? Describe one thing that would go wrong with Normal memory.
4. Where did 64 bits break Turk-OS despite Task 16.9, and why could the compiler not warn about it?
5. Why do `ldaxr` and `stlxr` need the MMU and the caches on?
6. After `svc`, the kernel does not add 4 to the return address; after `brk` it does. Why the difference, and how does RISC-V handle `ecall`?
7. Why does AArch64 need no TSS and no `sscratch` to find the kernel stack?
8. Which of the two ports was the most work, and what does that tell you about the architecture interface?
9. How can one TurkFS image serve three architectures, and what change to the format would break that?

7 Going further

Bring up the other cores: PSCI `CPU_ON` (a firmware call through `hvc` on QEMU's default `virt`), per-CPU data in `TPIDR_EL1`, SGIs for rescheduling and TLB shootdown, and your Phase 9 spinlocks finally spinning on real contention. Move to GICv3 (`gic-version=3`), whose CPU interface is a set of system registers instead of MMIO. Give each process an ASID in the top bits of `TTBR0_EL1`, set `nG` on user pages, and stop flushing the whole TLB on every switch; measure the context-switch benchmark before and after. Try the 16 KiB and 64 KiB granules and see how the walk changes. Run Turk-OS under KVM on an ARM machine. Port it to the Raspberry Pi 5, whose peripherals sit behind the RP1 chip and whose standard Linux kernel uses 16 KiB pages. Or take Phase 15's Track D: now that the generic code is 64-bit clean, x86-64 is mostly a new `kernel/arch/x86_64`.

8 Closing

Fourteen months ago Turk-OS was a boot sector that printed one line. Now one source tree builds a kernel and a userland for three processor architectures, two of which you met only in this stage, and the same disk image boots on all of them. Porting showed you which parts of your kernel were ideas and which were accidents of the PC: the scheduler, the file system and the system call layer did not change, while the code that knew about descriptors, rings and port I/O moved into one directory per machine. That separation, more than any single feature, is what makes an operating system last. Keep the journal, keep the tests green, and if Timur-RV32IMC runs your kernel one day, write it down.

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.

Port references used in Stage V

XV6 Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*, RISC-V edition. The companion book to MIT's xv6-riscv kernel. Free at https://pdos.csail.mit.edu/6.1810/; source at https://github.com/mit-pdos/xv6-riscv.

RISCV *The RISC-V Instruction Set Manual*, Volume I (Unprivileged ISA) and Volume II (Privileged Architecture); *RISC-V Supervisor Binary Interface Specification*. Free at https://riscv.org/specifications/ratified/ and https://github.com/riscv-non-isa/riscv-sbi-doc.

ARM *Arm Architecture Reference Manual for A-profile architecture* (DDI 0487), and Arm's *Learn the architecture* guides: AArch64 exception model, AArch64 memory management, Generic Timer, GIC. Free at https://developer.arm.com/documentation.

SPEC *Devicetree Specification* (https://www.devicetree.org/specifications/); *Virtual I/O Device (VIRTIO)*, OASIS (https://docs.oasis-open.org/virtio/virtio/); the QEMU documentation for the RISC-V and Arm `virt` boards (https://www.qemu.org/docs/master/system/).

All four are free. Specifications are revised often, so the reading plans cite them by chapter and section title rather than by page.